Faulty Barcode Detection

Tewson Seeoun
4922783735

INTRODUCTION

A barcode (figure 1-a) is a set of lines with different thickness representing a set of numbers. It is a very popular way to assigned unique identification to objects. Barcodes are popular because they are easy to be read by a barcode reader. The barcode reader projects a stripe of red light on the barcode, measure the reflection and convert the values to numbers. The brief idea of barcode reading is that the reader tries to measure the thickness of the bars and compare it to a defined standard. Therefore, due to the narrowness of the gaps between the bars, small stains (figure 1-b) could cause some error in reading.

According to a printing company's demand, there should be an automatic way to detect from a photograph whether a printed barcode is clear enough to be read. The author proposes a combination of techniques as an algorithm and has implemented a prototype software, which will be discussed thoughout this report.

PRINCIPLES

One important characteristic of barcodes is: it contains many straight lines. This idea could be used to locate the barcode because this pattern of lines rarely appear elsewhere. After the barcode is located, the stains need to be detected. To detect the stains, the author proposes an idea of comparing input barcode image with the ideal, or clean, barcode image. After comparing, the result is classified with a threshold of the discovered error. This procedure is depicted as a flowchart in Figure 2.

(a) (b)

Figure 1. Barcodes
(a) Example of barcode (b) Barcode with stains

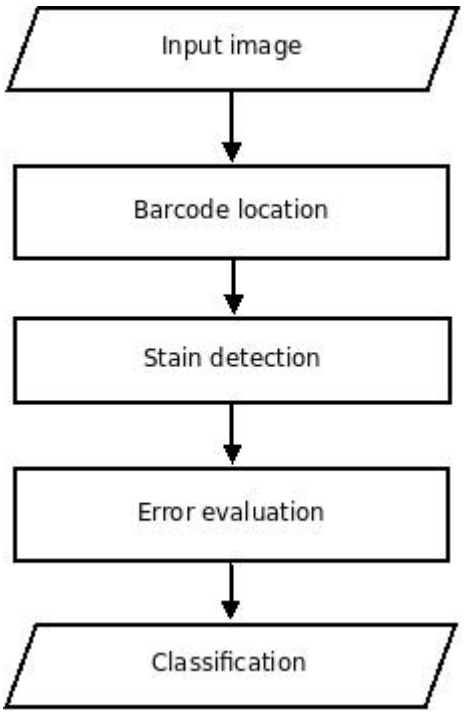


Figure 2. Brief flowchart of the algorithm

Barcode Location

As mentioned above, to locate the barcode, the system detects a pattern of packed lines. To achieve that, the proposed algorithm uses Hough transform, which maps the image pixel coordinates to distances form origin and angles, or the parameter plane. The desired ordinary shapes could be found from considering this transformed image: the accumulated bright spots denote lines. The whole process of barcode location is depicted in Figure 4.

In details, before using Hough transform, the input image needs to be converted into a binary, or black and white, image. An example of line detection is shown in Figure 3-a and Figure 3-b.

(a) (b)

Figure 3. (a) Input image (b) Line detection

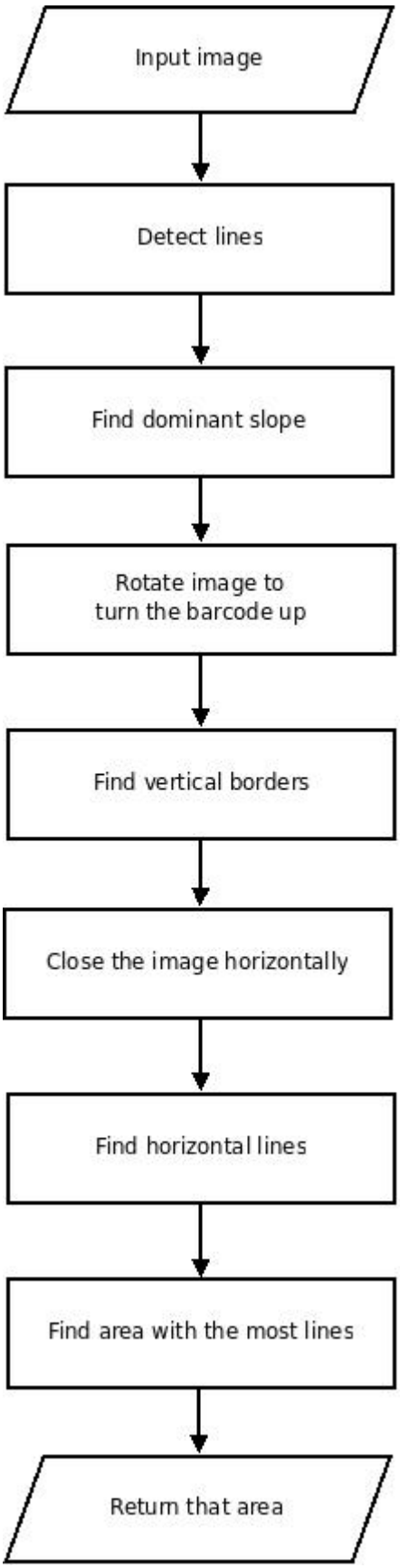
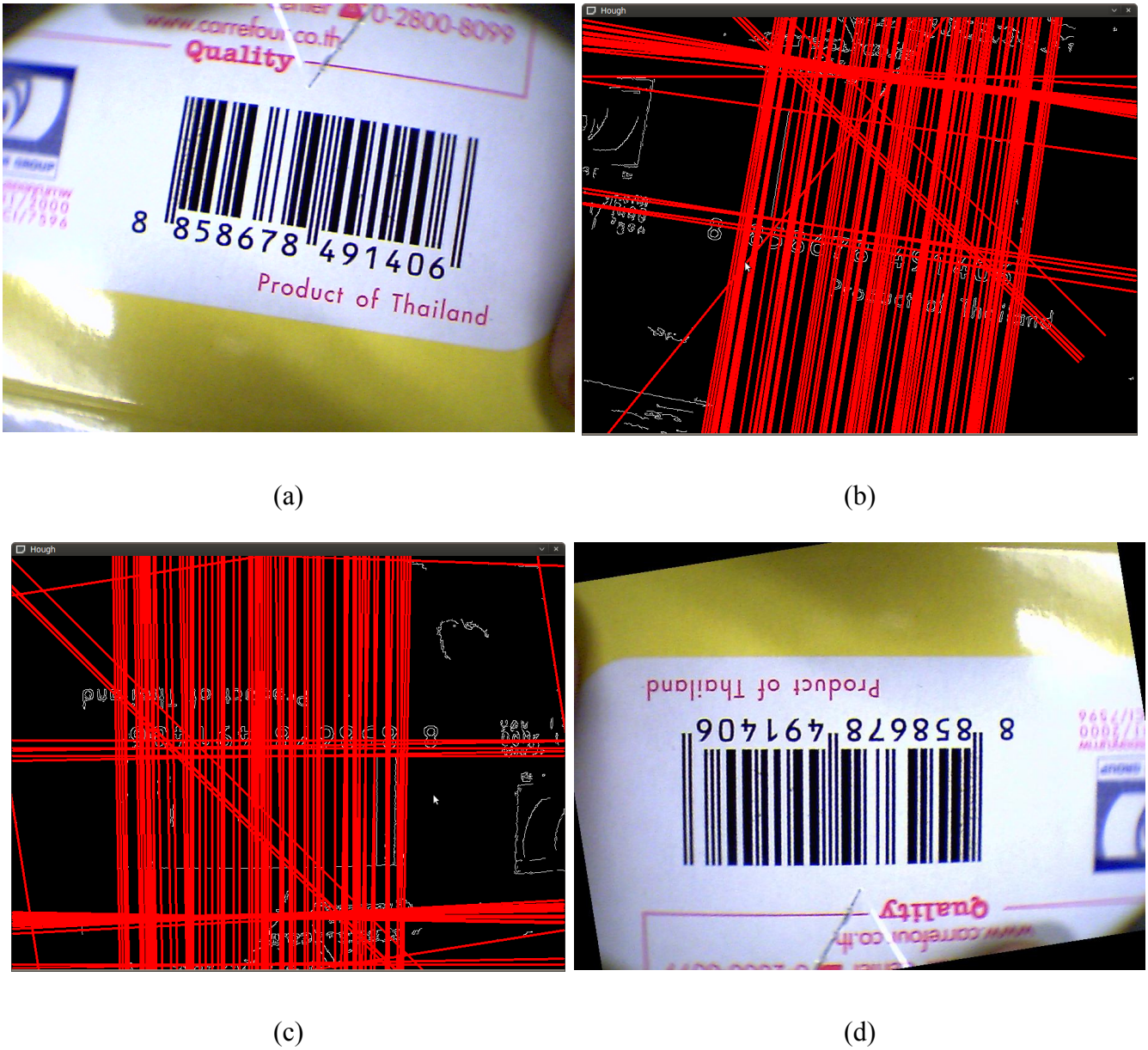


Figure 4. Flowchart for barcode location

In practice for line detection, an adjustable threshold is needed to consider how “bright” the spots in the parameter plane has to be in order to be counted as a line. This threshold is obtained by trial-and-error method.

In the proposed algorithm, the barcode is easier to process if it directs vertically. Therefore the image needs to be rotated to the right direction. To achieve this, slope angles are extracted from all the detected lines. These angles are then quantized into an appropriate number of levels. The “voted” quantized level is chosen and is average to be a dominant angle of the barcode. This dominant angle is used to rotate the image. This rotation is shown Figure 5-a, 5-b, 5-c, and 5-d.



(a) (b)

(c) (d)

Figure 5. Image rotation
(a) Input image (b) Line detection (c) Image rotated according to lines (d) Rotated image

After rotation, the system then finds the boundaries of the barcode. It starts with finding the furthermost vertical lines (This introduces several limitations of the system which will be discussed later on). These vertical boundaries are lines that possess the steepest slopes. The process of finding the vertical boundaries is shown in Figure 6.

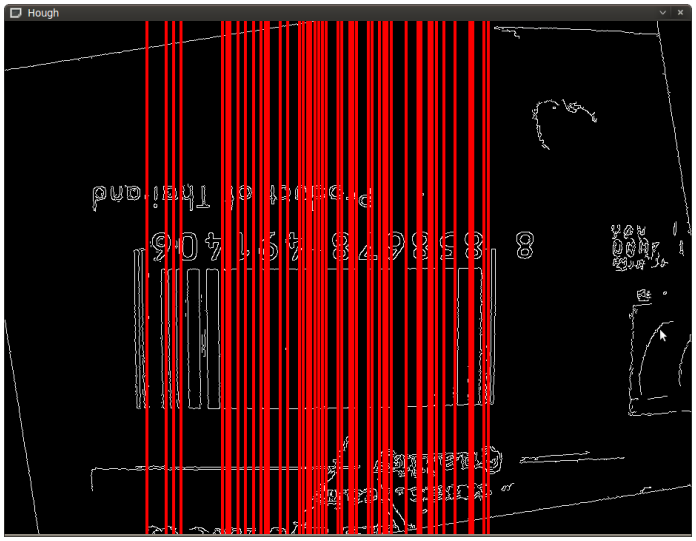


Figure 6. Vertical lines

After the vertical boundaries are discovered, the system then finds the horizontal boundaries. A problem arises here: there are no such horizontal lines detected in some barcodes. To circumvent this problem, the system then dilates the image horizontally to bridge the gaps between the bars, and the horizontal boundaries could be determined. However, the system could not choose the furthermost horizontal boundaries as it does for vertical ones, because there might be some vertical lines that don't bound the barcode, such as the ones appear from connected numbers below the barcode. Therefore, the system uses the vertical boundaries and the newly-acquired horizontal lines to define grid regions, and find the region with most lines. Now the barcode is located. This process is shown in Figure 7.

(a) (b) (c)

Figure 7. Barcode location: finding region with most lines
(a) Grid regions (b) Located barcode (c) Located barcode highlighted with red border

Stain Detection

After the barcode is located, the system tries to detect the stains on the barcode. By observation, these stains occur as gray spots between the bars. To detect them, the proposed algorithm tries to compare the barcode image with its ideally clean version. The details are describe as a flowchart in figure 8.

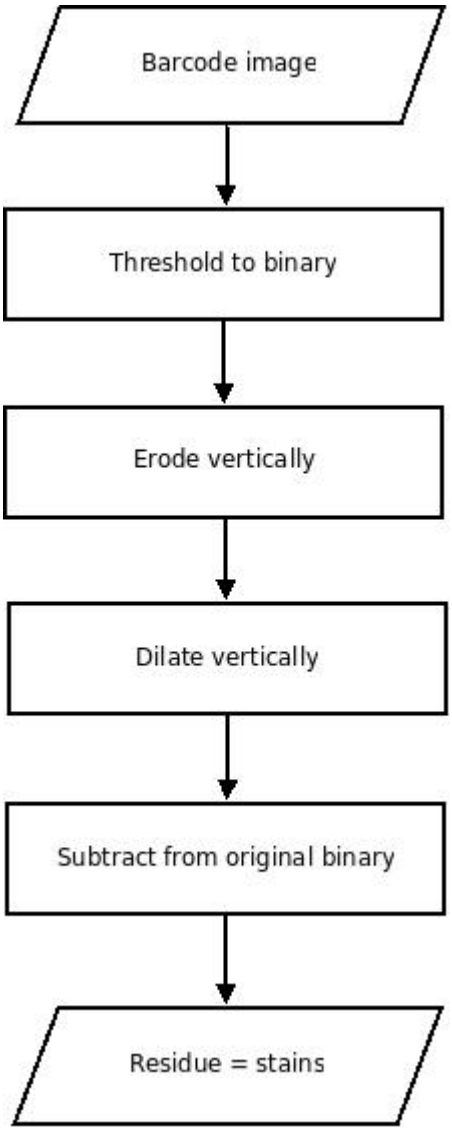
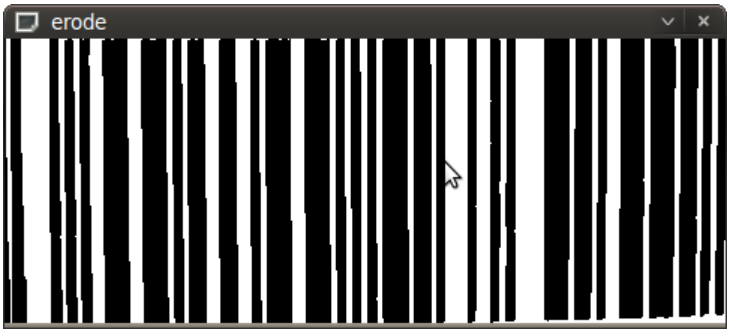


Figure 8. Flowchart for stain detection

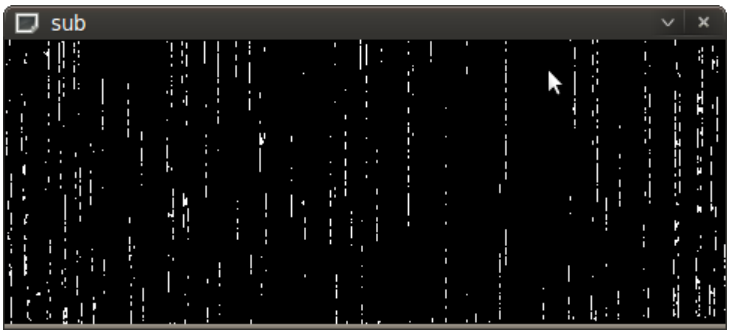
To get through this process, which is depicted in Figure 9, the barcode image is converted into a binary image to get rid of blurry details, such as those in JPEG images, that might cause errors. The image is converted with a high threshold number to keep stains. There are two copies of this binary image so the system could compare both after the process. One binary image is then eroded with a vertical structuring element, to get rid of the stains. The proposed algorithm uses a vertical structuring element because it does not affect the bars much. After erosion, the image is dilated back to the almost same detail, with no more stains. This clean image is then subtracted from the original binary one. The residue is the stains, because the only difference of the two should be the presence of faulty ink.

(a)

(b)



(c)

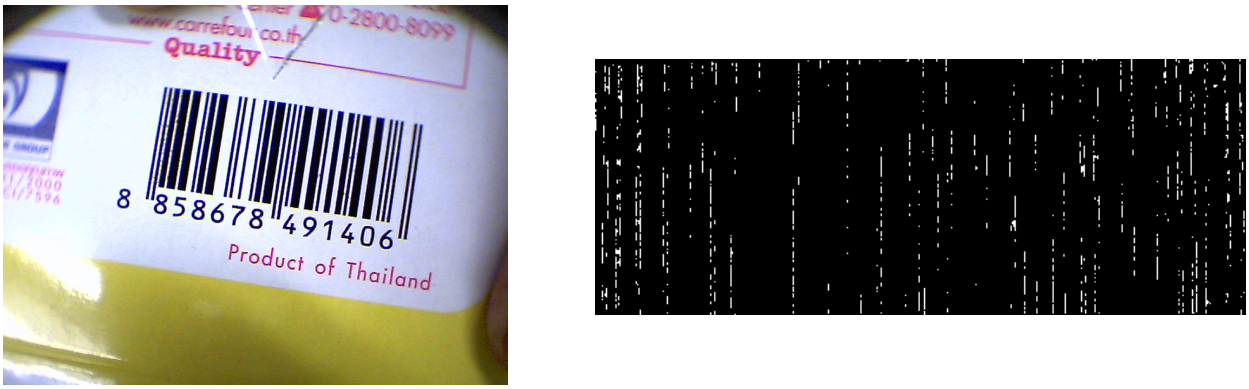


(d)

Figure 9. Stain detection
(a) Barcode image
(b) Binary version
(c) After erosion and dilation
(d) Residue from subtraction

(a)

(b)



(c)

Figure 10. Barcode images and their stain traces (*small differences might be hard to notice*)
(a) Clean barcode, 0 dirty blocks found
(b) Rotated clean barcode, 19 dirty blocks found
(c) Rotated faulty barcode, 131 dirty blocks found

Error Evaluation

After obtaining the trace of the stains, the system uses this to measure how "dirty" the barcode image is. By observation, most trace images, either clean or dirty, consist of many white short vertical lines. These lines are caused by image rotation in barcode location. However, these white lines from clean images have the width of no larger than one pixel. Therefore, to measure the dirtiness, the proposed algorithm tries to count horizontal white blocks, so-called "dirty blocks", with the height of one pixels and the width of two pixels. Figure 10 compares occurrence of the dirty blocks in different barcode images.

To classify barcode images according to the stain trace images, there must be enough number of already-classified data set. Therefore, the trend of the number of the dirty blocks could be found. This number is then set as a threshold to distinguish between clean and faulty unseen barcode images.

From the current small data set (8 images), a rough threshold could be set at 50 dirty blocks.

An experiment is also conducted to measure the system's robustness. The author compares a barcode image (Figure 11-a), a blurred version (with 5-px gaussian blur – Figure 11-b), and a downsampled version (50% in size – Figure 11-c), to test focusing problem and scaling problem. **There is no need to test for rotation problem since Hough transform is used at the first place**

The blurred barcode image yields dropped performance. The dirty blocks detected reduce from 131 blocks to 121 blocks, which makes sense, because human eye would also see no stains on a blurred barcode image.

Downsampling also makes the performance drop, from 131 blocks to 92 blocks. This can also be explained as in the case of blurring.

(a) (b) (c)

Figure 11. Robustness of the system
(a) Original image, dirty blocks detected = 131 blocks
(b) Blurred image, dirty blocks detected = 121 blocks
(c) Downsampled image, dirty blocks detected = 92 blocks

IMPLEMENTATION

The prototype software is written in Python programming language and is tested on Ubuntu Linux operating system. It uses its own image processing library, Python Image Library (PIL), and an open source library from Intel corporation called "OpenCV". The author chose Python as the main tool because of its easiness. Important functions used in the software is listed below.

lines cvHoughLines2(image, lineStorage, method, rho, theta, threshold, param1, param2)
- detect lines and return those lines

cvCreateStructuringElementEx(cols, rows, anchor_x, anchor_y, shape, values (optional))
- create a structuring element to be used in erosion or dilation

cvErode(src,dst,element,iteration)
- erode an image from source to destination with a desired structuring element

cvDilate(src,dst,element,iteration)
- dilate an image from source to destination with a desired structuring element

cvGetSubRect(src,cvRect(left,top,width,height))
- crop a rectangular area from an image

cvThreshold(src,dst,threshold,maximum_value,threshold_type)
- convert source image into binary image according to the given threshold value

Apart from the source code, the software requires these following software packages in Ubuntu:

python-opencv and *python-pil*

The software requires only one input, the image file path. The pattern of using it is:

python findbarcode.py FILE_PATH

DISCUSSION

In the beginning, the implementation process was slow. It was stuck at the barcode location part. Locating the exact position of the barcode is complicated. The proposed algorithm of finding the furthermost vertical boundaries is fast and easy to implement, but it introduces several limitations: there should be as few as possible other lines with the same direction as the barcode's apart from the barcode itself in the image, and the image should not be distorted or skew, in other words, it has to be a frontal image of a barcode on a planar object. The quality of the image should also be concerned, for example, lighting condition and size.

Along implementation, there were not many problems. The software is robust to image rotation and is able to locate the barcode (without the limitations mentioned above). The computational cost is not so high and the processing is fast, at around 1.5 seconds per image (depending on image size).

There are several adjustable values in the system, such as, threshold value of Hough transform, minimum size required for input images, number of quantizing levels for slope angles, structuring element for dilation and erosion, or threshold value of converting images to binary images. These values should be adjusted depending on the scope of work, which could be different place to place.

Further improvement could be noticed if there were a larger data set. The algorithm could be more robust with different approaches, for example, using pattern recognition techniques such as Haar-like features in barcode location, or using self-organizing feature map to distinguish the packed barcode lines from other lines.

CONCLUSION

Faulty barcode detection could be done automatically through three steps: barcode location, stain detection, and error evaluation. In barcode location, the proposed algorithm detects lines, choose the dominant angle from the lines, which denote the direction of the barcode, and rotates the image upward accordingly. It then finds the vertical and horizontal boundaries. The region with most lines is considered as the region containing the barcode. The barcode is then converted into a binary image. The stains in this binary image is cleaned off through erosion and dilation and then subtracted from the original binary image. The residue from subtraction is examined to measured the level of dirtiness. If this level exceeds a threshold value, this barcode is classified as faulty.

To complete this system, a larger data set is needed. Therefore, a threshold model could be set and used to classify unknown barcode images.

DEMONSTRATION

For the software's source code and a demo video, please visit *http://tewson.com/barcode* .